

Industrial Internship Report on

"URL Shortener"

Prepared by

Puralasetti Khayati Sri

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was "URL Shortener"

This report presents the work completed during the six-week industrial internship through upskill Campus and The IoT Academy in collaboration with UniConverge Technologies Pvt Ltd (UCT). The project developed during the internship was a **URL Shortener**, a Python-based web application that converts long URLs into short and manageable links.

The application uses Python and Flask to process URLs and SQLite to store the mapping between original URLs and unique short codes. It generates shortened URLs and redirects users to the original URL when the shortened link is accessed. The project provided practical experience in web development, database integration, URL processing, testing, debugging, and problem-solving.

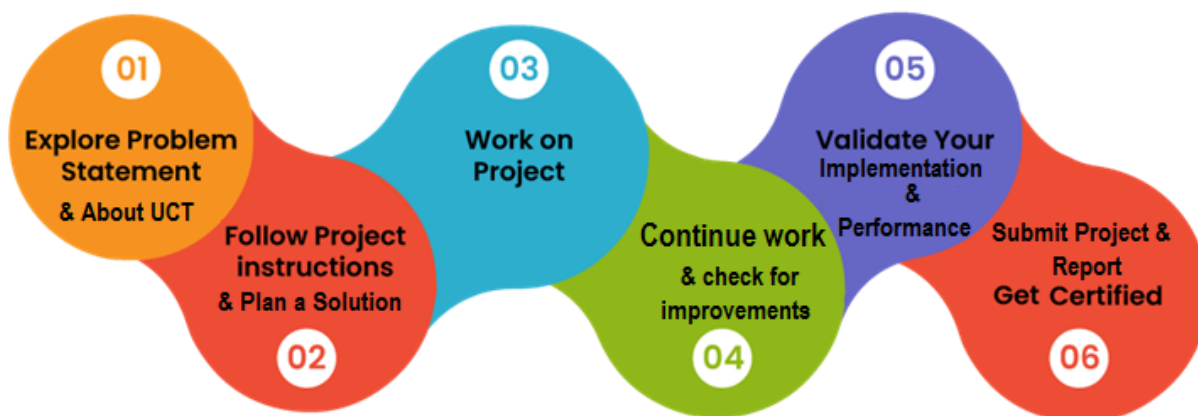
This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	4
2.1	About UniConverge Technologies Pvt Ltd	Error! Bookmark not defined.
2.2	About upskill Campus	8
2.3	Objective	10
2.4	Reference	10
2.5	Glossary.....	10
3	Problem Statement	11
4	Existing and Proposed solution	12
5	Proposed Design/ Model	14
5.1	High Level Diagram (if applicable)	14
5.2	Low Level Diagram (if applicable)	15
5.3	Interfaces (if applicable)	15
6	Performance Test	16
6.1	Test Plan/ Test Cases	16
6.2	Test Procedure	17
6.3	Performance Outcome	17
7	My learnings.....	18
8	Future work scope	19

1 Preface

The six-week Python Internship Program provided me with valuable practical exposure to Python programming and software development. During the internship, I worked on various projects involving problem-solving, file handling, APIs, automation, and application development, which helped me apply my academic knowledge to real-world problems.



The internship was planned in a structured manner, progressing from learning fundamental concepts to developing, testing, and improving projects. It enhanced my technical skills, logical thinking, debugging abilities, and confidence in developing Python-based solutions.

I sincerely thank the USC/UCT program coordinators, mentors, faculty members, and everyone who supported and guided me throughout the internship. I am grateful for this learning opportunity and encourage my juniors and peers to actively participate in internships to gain practical knowledge and prepare themselves for their future careers.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



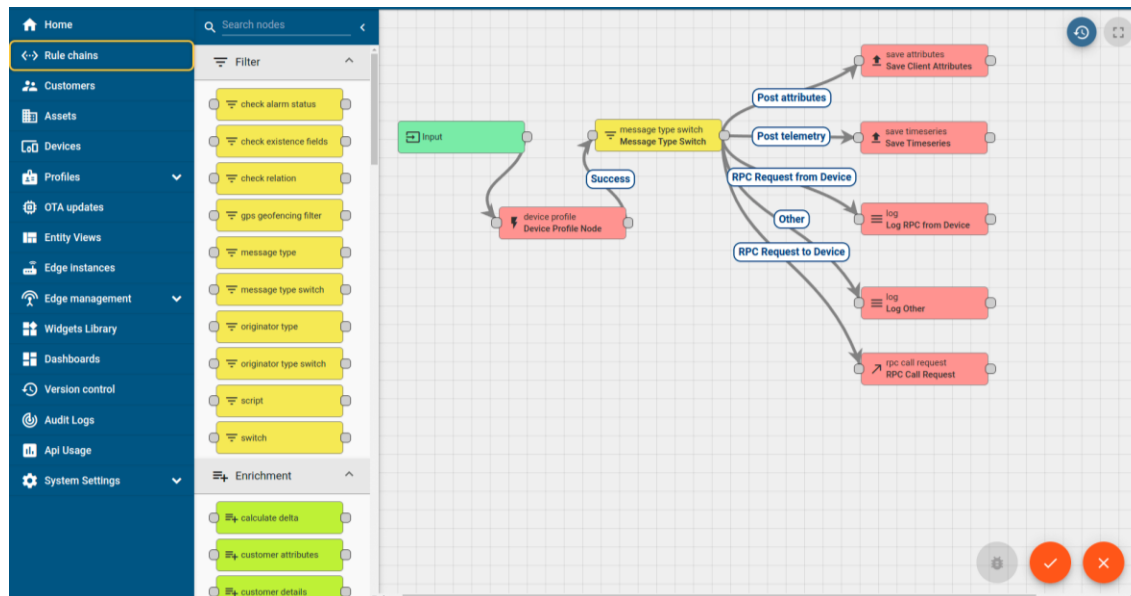
i. UCT IoT Platform ()

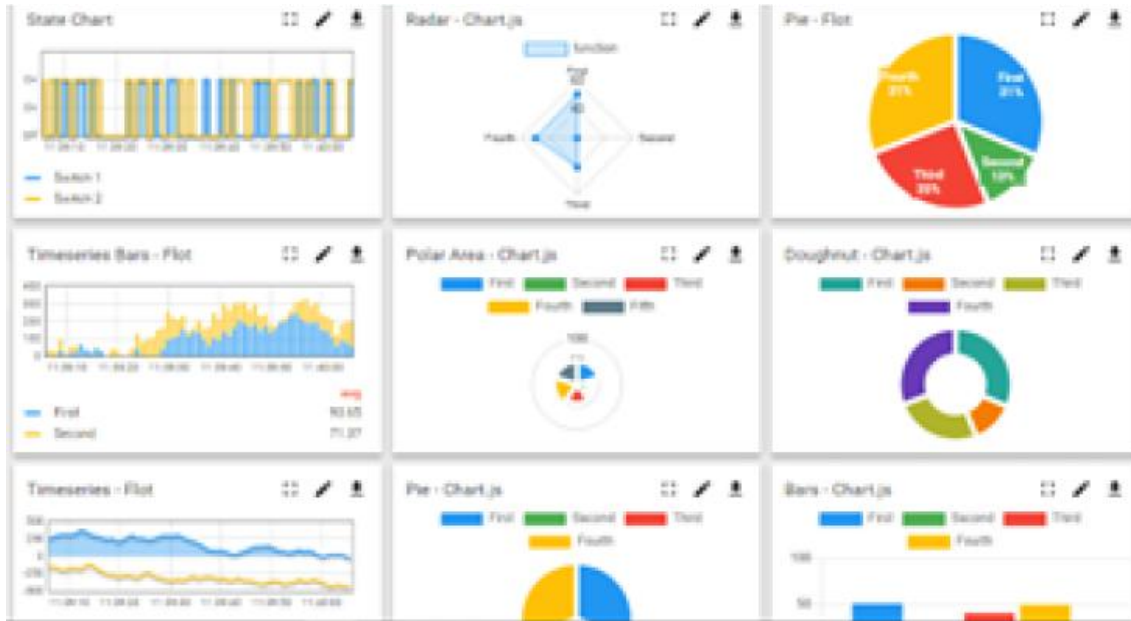
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





FACTORY WATCH

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleashed the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



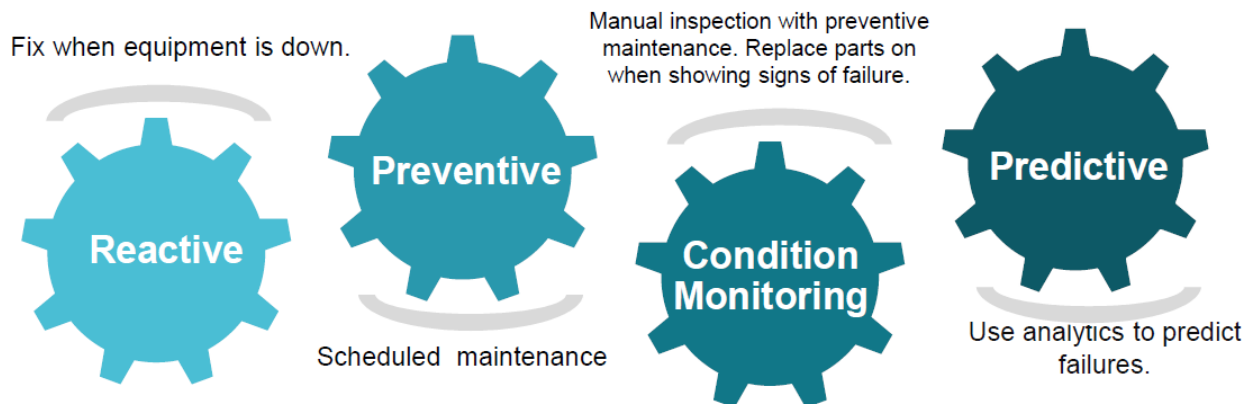


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

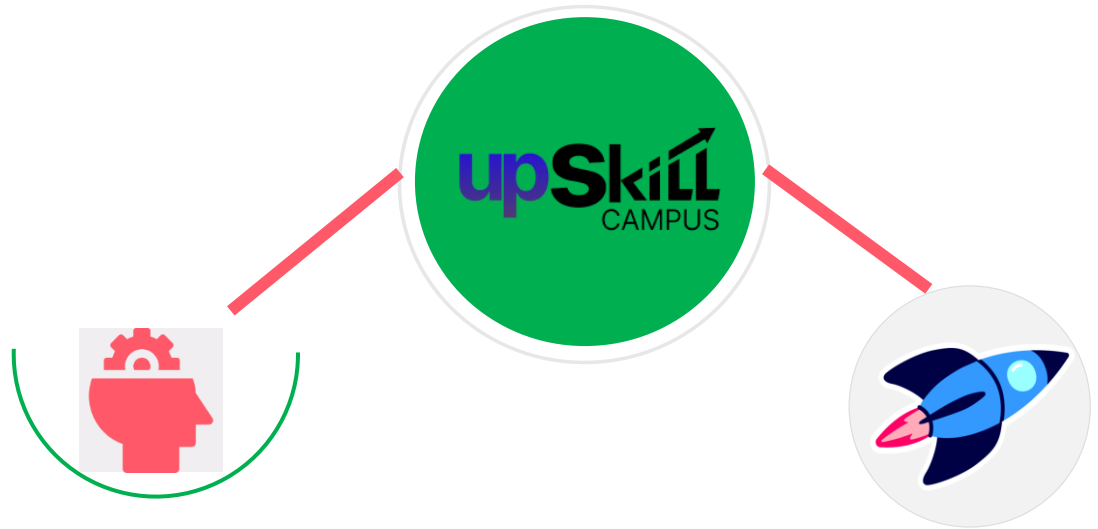
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.1 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

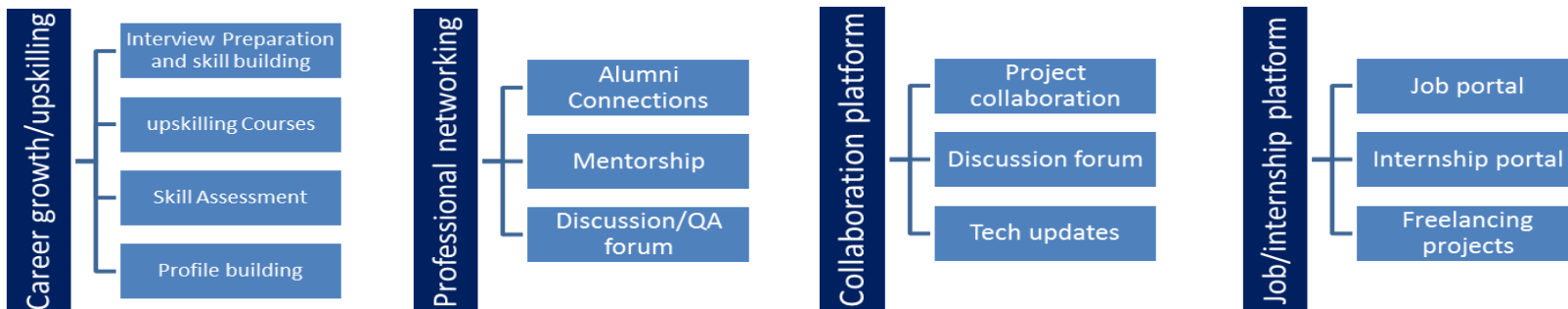
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>



2.2 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.3 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.4 Reference

- [1] <https://docs.python.org/3/>
- [2] <https://flask.palletsprojects.com/>
- [3] <https://openweathermap.org/api>

2.5 Glossary

Acronym	Terms
API	Application Programming Interface
IDE	Integrated Development Environment
URL	Uniform Resource Locator
JSON	JavaScript Object Notation

3 Problem Statement

Long URLs can be difficult to share, remember, and use, especially when they contain lengthy parameters or complex paths. They can also make messages, documents, and other forms of communication look unnecessarily complicated.

The objective of this project is to develop a **URL Shortener application** that accepts a long URL from a user and generates a shorter, unique URL. The shortened URL should be stored along with the original URL in a database.

When a user accesses the shortened URL, the application should identify the corresponding original URL from the database and automatically redirect the user to it.

The system therefore needs to provide:

- A user interface for entering long URLs.
- A mechanism for generating unique short codes.
- Database storage for original and shortened URLs.
- A mechanism for retrieving the original URL.
- Redirection from the shortened URL to the original URL.
- Basic validation and error handling.

The overall workflow is:

**Long URL → Generate Short Code → Store in Database → Generate Short URL → User
Accesses Short URL → Retrieve Original URL → Redirect**

4 Existing and Proposed solution

Existing Solution

Existing URL-shortening services provide users with shortened links by using a server-side system to generate unique identifiers and maintain mappings between shortened and original URLs.

However, using an external URL-shortening service may provide limited understanding of how the underlying URL-shortening process works. Users may also have limited control over the database, application logic, and customization of the service.

Proposed Solution

The proposed solution is a simple Python-based URL Shortener web application developed using **Flask and SQLite**.

The application provides a user interface where users can enter a long URL. The backend processes the URL, generates a unique short code, and stores the original URL and short code in the SQLite database.

The shortened URL can then be accessed by the user. When the short URL is requested, Flask retrieves the corresponding original URL from the database and redirects the user to the original website.

Proposed System Workflow

1. User enters a long URL.
2. Application receives and validates the URL.
3. A unique short code is generated.
4. Original URL and short code are stored in SQLite.
5. The application generates the shortened URL.
6. User accesses the shortened URL.
7. Flask searches the database for the corresponding short code.
8. Original URL is retrieved.
9. User is redirected to the original URL.

Value Addition

The proposed project provides:

1. Simple and user-friendly URL shortening.
2. Understanding of URL mapping mechanisms.

3. Local database management.
4. Unique short-code generation.
5. URL redirection.
6. Basic validation and error handling.
7. Practical experience in Python web development.

4.1 Code submission (Github link)

https://github.com/khayatisri25/upskillcampus/blob/main/URL_Shortener.py

4.2 Report submission (Github link) :

https://github.com/khayatisri25/upskillcampus/blob/main/URL_Shortener_Puralasetti_Khayati_Sri_USC_UCT.pdf

5 Proposed Design/ Model

The proposed URL Shortener consists of three major components:

1. User Interface

The user interface allows the user to enter a long URL and submit it for shortening. The generated shortened URL is displayed to the user.

2. Flask Backend

The Flask backend receives the URL, processes the request, generates a unique short code, communicates with the database, and handles redirection.

3. SQLite Database

The SQLite database stores the mapping between the original URL and its corresponding short code.

5.1 High Level Diagram

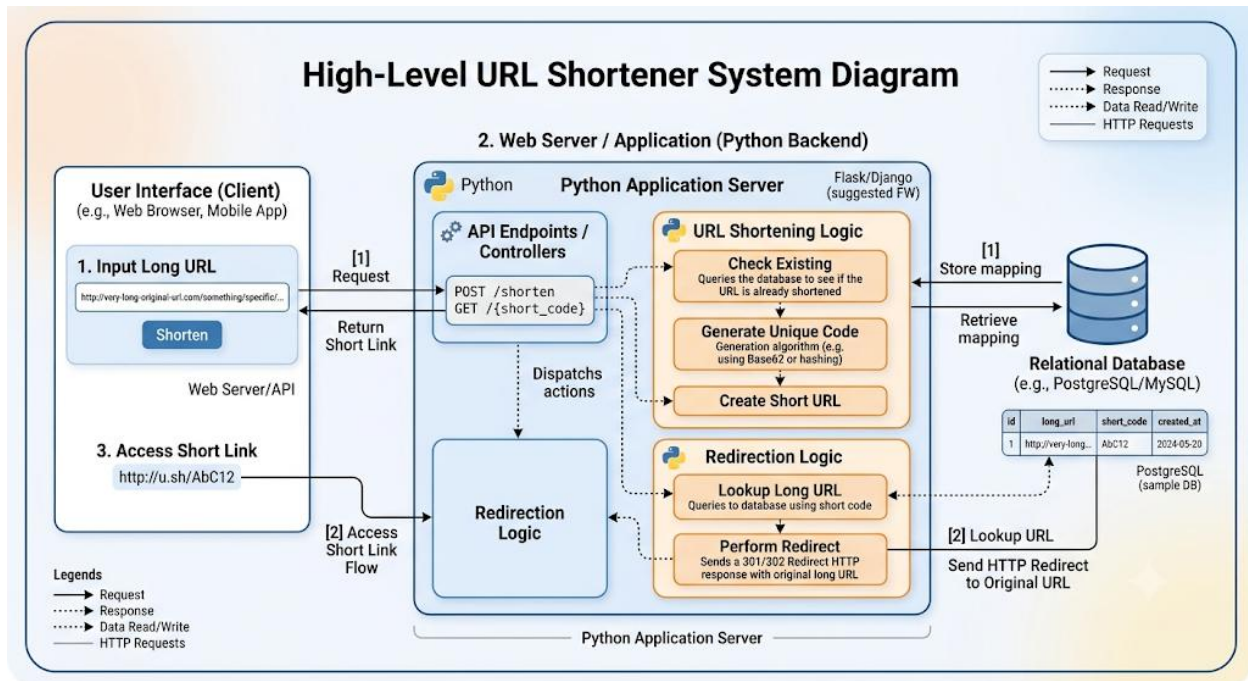
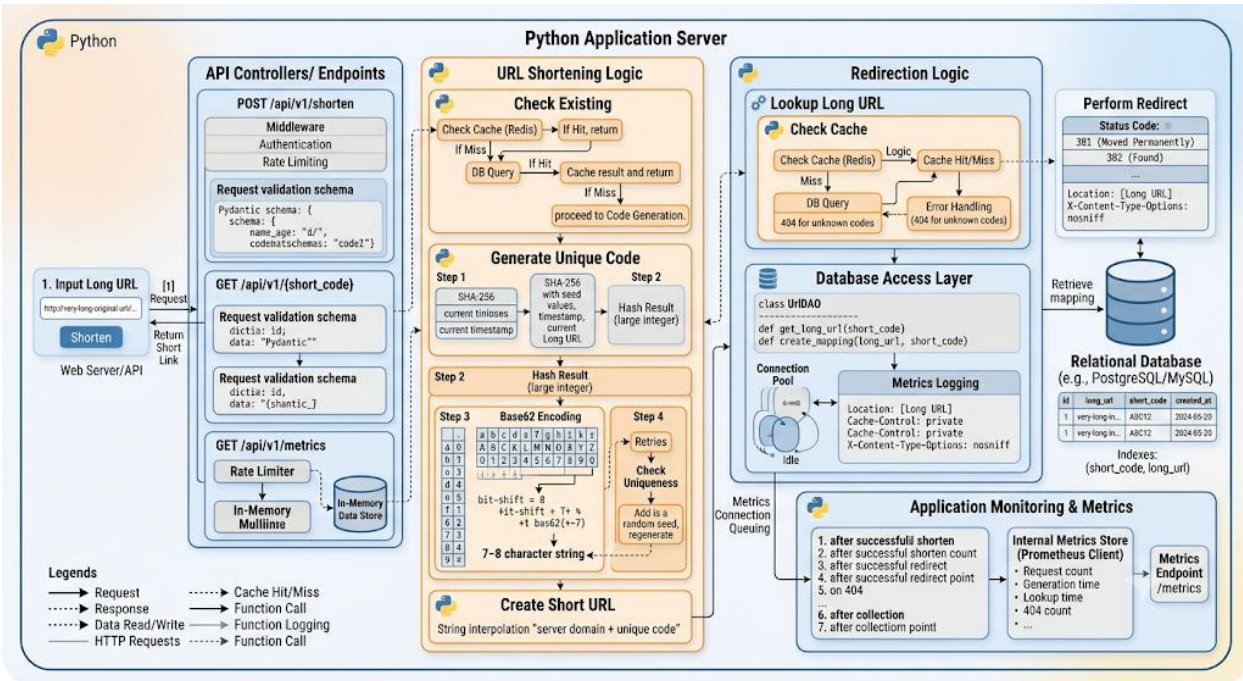
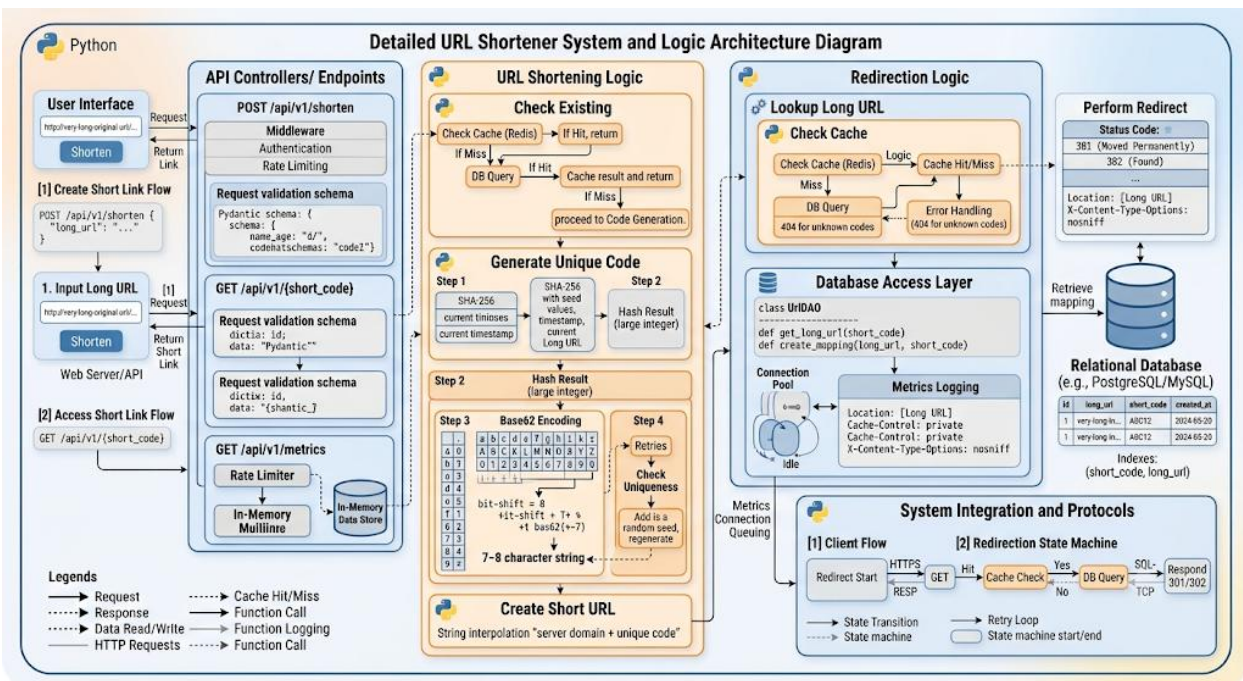


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram



5.3 Interfaces



6 Performance Test

Performance testing was conducted to verify whether the major components of the URL Shortener were functioning correctly.

The important constraints considered were:

- Correct URL processing.
- Unique short-code generation.
- Correct database storage.
- Correct URL retrieval.
- Successful redirection.
- Handling of invalid URLs.
- Handling of unavailable short codes.
- Application response and stability.

6.1 Test Plan/ Test Cases

Test Case	Input	Expected Result	Status
TC01	Valid long URL	Short URL generated	Pass
TC02	Multiple different URLs	Unique short codes generated	Pass
TC03	Valid short URL	Redirects to original URL	Pass
TC04	Invalid URL	Validation/error message displayed	Pass
TC05	Empty URL	User prompted to enter URL	Pass
TC06	Non-existing short code	Error/not-found response	Pass
TC07	Database insertion	URL mapping stored successfully	Pass
TC08	Database retrieval	Original URL retrieved correctly	Pass

6.2 Test Procedure

The application was tested by entering different valid and invalid URLs through the user interface. For valid URLs, the generated short codes were checked for uniqueness and the corresponding records were verified in the SQLite database.

The generated shortened URLs were then accessed through the browser to verify whether they correctly redirected to the original URLs.

Invalid inputs, empty inputs, and unavailable short codes were also tested to verify the application's error-handling functionality.

6.3 Performance Outcome

The testing demonstrated that the major functionalities of the URL Shortener were working as intended. The application successfully processed long URLs, generated shortened links, stored URL mappings in the SQLite database, retrieved original URLs, and performed redirection.

Testing also helped identify and resolve issues related to input validation, database operations, short-code uniqueness, and URL routing.

7 My learnings

The six-week URL Shortener project provided practical experience in Python-based web application development.

The major technical learnings include:

- Improved Python programming skills.
- Learned practical Flask web development.
- Understood Flask routes and request handling.
- Learned database operations using SQLite.
- Understood how URL mappings are stored and retrieved.
- Learned the concept of unique short-code generation.
- Understood HTTP redirection.
- Practiced URL validation and error handling.
- Gained experience in debugging and testing.
- Improved understanding of frontend-backend communication.
- Learned how to structure a small web application.

8 Future work scope

Although the basic URL Shortener functionality was successfully implemented, several additional features can be developed in the future.

Possible Future Enhancements

1. **Custom Short URLs**
Allow users to choose their own short codes.
2. **Click Tracking**
Track the number of times each shortened URL is accessed.
3. **Analytics Dashboard**
Display statistics related to URL usage.
4. **User Authentication**
Allow users to create accounts and manage their shortened URLs.
5. **URL Expiration**
Provide an option to automatically expire shortened URLs after a specified period.
6. **QR Code Generation**
Generate QR codes for shortened URLs.
7. **Improved Security**
Add stronger URL validation and protection against malicious URLs.
8. **Cloud Database**
Replace the local SQLite database with a cloud-based database for scalability.
9. **Deployment**
Deploy the application on a web hosting platform so that it can be accessed online.
10. **API Development**
Develop a REST API so that other applications can use the URL-shortening functionality.

